Walchand College of Engineering, Sangli

Department of Computer Science and Engineering

Class: Final Year B.Tech(Computer Science and Engineering)

Year: 2025-26

Semester: 1

Course: High Performance Computing Lab

Practical No. 2

PRN: 22510019

Name: Soham Patil

Batch: B6

Title of practical: Study and implementation of basic OpenMP clauses

Implement following Programs using OpenMP with C:

1. Vector Scalar Addition
2. Calculation of value of Pi

Analyse the performance of your programs for different number of threads and Data size.

Problem Statement 1:

Code snapshot :

```c
void vector_scalar_addition(float *a, float scalar, int n, int num_threads)
{
    #pragma omp parallel for num_threads(num_threads)
    for (int i = 0; i < n; i++)
    {
        a[i] += scalar;
    }
}

int main() {
    int n;
    float scalar;
    int num_threads;

    printf("Enter size of vector: ");
    scanf("%d", &n);

    printf("Enter scalar value to add: ");
    scanf("%f", &scalar);

    printf("Enter number of threads: ");
    scanf("%d", &num_threads);

    float *a = (float *)malloc(n * sizeof(float));

    for (int i = 0; i < n; i++)
    {
        a[i] = i;
    }

    double start = omp_get_wtime();
    vector_scalar_addition(a, scalar, n, num_threads);
    double end = omp_get_wtime();

    printf("Execution Time: %f seconds\n", end - start);
```

Output :

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$ ls
ps1.c
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$ gcc -fopenmp ps1.c -o ps1
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$ ./ps1
Enter size of vector: 10
Enter scalar value to add: 25
Enter number of threads: 7
Execution Time: 0.002311 seconds
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$
```

Information:

The provided C program performs vector-scalar addition using OpenMP for parallelization. It takes the vector size, scalar value, and number of threads, then adds the scalar to each element of the vector in parallel. The use of OpenMP's parallel directive allows the computation to be distributed across multiple threads, improving performance for large vectors.

Analysis:

Running the program with a vector size of 10, scalar value 25, and 7 threads resulted in an execution time of just **0.002311 seconds**.

This shows that even for small vectors, OpenMP can efficiently utilize available CPU cores, though the parallel overhead may outweigh the benefits for very small data sizes. The program is easy to scale for larger vectors, where the performance gains from parallelization would be more significant.

Problem Statement 2:

Code snapshot :

```c
#include <stdio.h>
#include <omp.h>

double compute_pi(int num_steps, int num_threads)
{
    double step = 1.0 / (double)num_steps;
    double sum = 0.0;

    #pragma omp parallel for reduction(+:sum) num_threads(num_threads)
    for (int i = 0; i < num_steps; i++)
    {
        double x = (i + 0.5) * step;
        sum += 4.0 / (1.0 + x * x);
    }
    return step * sum;
}

int main() {
    int num_steps;
    int num_threads;

    printf("Enter number of steps: ");
    scanf("%d", &num_steps);
    printf("Enter number of threads: ");
    scanf("%d", &num_threads);

    double start = omp_get_wtime();
    double pi = compute_pi(num_steps, num_threads);
    double end = omp_get_wtime();

    printf("Calculated Pi = %.5f\n", pi);
    printf("Execution Time: %f seconds\n", end - start);

    return 0;
}
```

Screenshots:

```
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$ gcc -fopenmp ps2.c -o ps2
soham@Vivobook16x-Soham:/mnt/d/SEM VII/HPCL/Assignment2$ ./ps2
Enter number of steps: 12
Enter number of threads: 9
Calculated Pi = 3.14217
Execution Time: 0.001954 seconds
```

Information:

This C program estimates the value of Pi using numerical integration (the midpoint rule) and parallelizes the computation with OpenMP. The user specifies the number of integration steps and threads, and the program distributes the workload using OpenMP's parallel for with reduction, efficiently summing partial results from each thread.

Analysis:

With 12 steps and 9 threads, the program calculated Pi as **3.14217** in just **0.001954** seconds.

While the execution time is extremely low for small step counts, increasing the number of steps and threads generally improves both accuracy and speed.

However, for very small data sizes, the overhead of parallelization may outweigh its benefits, but for larger computations, OpenMP provides clear performance gains.

Github Link: **HPCL/Assignment2 at main · Somzee5/HPCL**

Final Year: High Performance Computing Lab 2025-26 Sem I